

RViz & Visualization Tools (RViz2, RQT plugins, URDF files)

Introduction to Robotics Lab

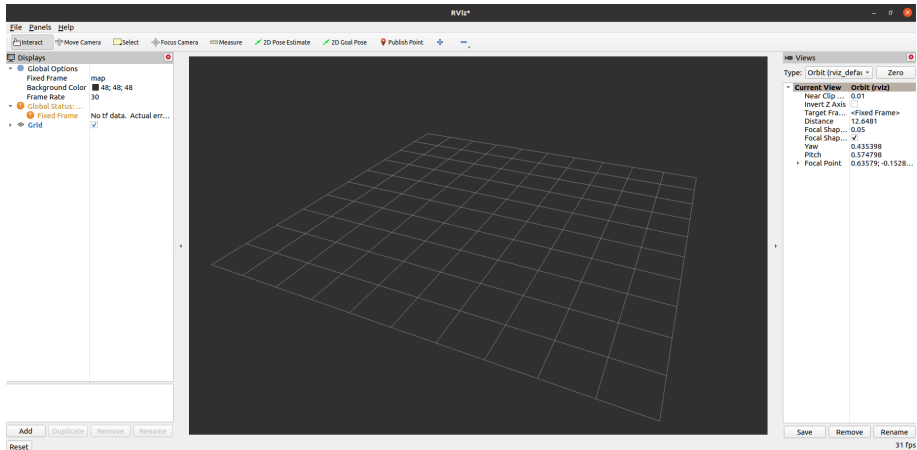
Konstantinos Asimakopoulos k_asimakopoulos@ac.upatras.gr
Lampros Printzios printzios_lampros@ac.upatras.gr

Department of Electrical and Computer Engineering
University of Patras

November 10, 2025

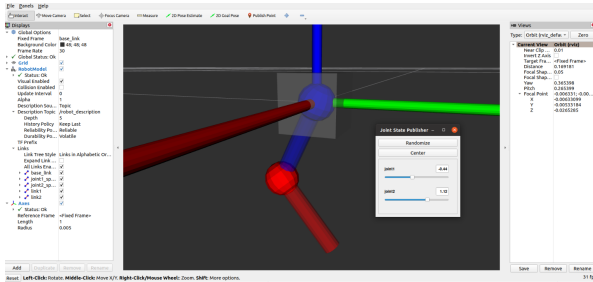
Table of Contents

- 1 RViz2
 - What is RViz2?
 - How it works?
- 2 RQT
 - What is RQT?
 - RQT plugins
 - rqt_publisher
 - rqt_topic
 - rqt_service_caller
 - rqt_graph
 - rqt_plot
 - rqt_tf_tree
 - rqt_image_view
- 3 URDF files
 - What is a URDF file?
 - Creating a URDF File
 - What is a XACRO file?
- 4 Double Pendulum Exercise
 - Create URDF
 - Visualize using RViz2
 - Our Executable Node
 - Launch File Overview
 - Inspect ROS2 Graph
 - Explaining ROS2 TF (Transform) Frame Tree
- 5 Elephant robotic arm Exercise
 - Create URDF
 - Visualize using RViz2
 - Our Executable Node
 - Launch File Overview
 - Inspect ROS2 Graph
- 6 Gazebo mention
- 7 References



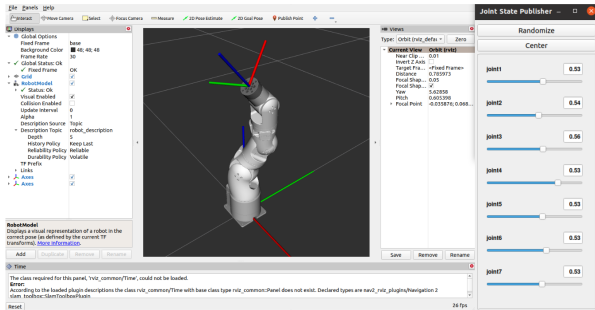
RViz2 starting Environment

RViz2



Double Pendulum RViz2

Elephant MyArm RViz2



RViz2 - What is RViz2?

- ① **RViz** (ROS Vizualization) is a **3D visualization tool** for the ROS framework. It helps visualizing and debugging a robot's state, TF frames, sensors, and environment data in a 3D environment.
- ② Key Features and Uses:
 - **3D Visualization & Debugging Tool**: show robot model's 3D view, perceive the environment, troubleshoot issues, ...
 - **Robot State & Navigation**: monitor and display robot state data (pose etc.), view navigation paths, ...
 - **Sensor Data**: display camera feeds, laser and LiDAR scans, maps' occupancy grids, ...
 - **Extensibility**: extend functionality with RQT and custom plugins, ...
- ③ In ROS2 we work with **RViz2**, which is the improved successor of RViz, built on the Qt5 application development framework.

RViz2 - How it works?

- ① **RViz** operates by subscribing to various data topics published within a ROS system. It then processes this information to produce real-time **3D visualizations** of the **robot** and its **environment**.
- ② Key Visualization Types:
 - **RobotModel:** displays the robot's 3D model using URDF descriptions
 - **Pose:** visualizes the robot's current pose (position and orientation) in 3D space
 - **TF (Transform Frames):** shows coordinate frames and their relationships within the TF tree
 - **Axes:** displays coordinate axes at specified positions to indicate reference frames and orientation
 - **PointCloud:** renders 3D point data from LiDAR, depth cameras, or other sensors for environmental perception
 - **Maps:** displays grid-based representations of the robot's environment
 - **Paths:** visualizes sequences of points, often representing planned routes
 - **Images:** shows camera feeds or other image-based data

Default - rqt

File Plugins Running Perspectives Help

Message Publisher

Topic	Type	Rate	Expression
/turtle1/cmd_vel	twist	1.00	
linear	geometry_msgs/Vector3		
x	double	1.0	
y	double	0.0	
z	double	0.0	
angular	geometry_msgs/Vector3		
x	double	0.0	
y	double	0.0	
z	double	0.0	

Topic Monitor

Topic	Value
/turtle1/rotate_absolute/action/status	not monitored
/turtle1/rotate_absolute/action/feedback	can not get message class for type "turt
/turtle1/pose	
y	5.544444561004639
x	11.088889122009277
theta	0.0
linear_velocity	1.0
angular_velocity	0.0
/turtle1/color_sensor	not monitored
/turtle1/cmd_vel	not monitored
/rosout	not monitored
/parameter_events	not monitored

Response

Field	Type	Value
-------	------	-------

MatPlot

Topic: /

Node Graph

Group: 10 Namespaces Actions tf Images Highlight Fit

Hide: Dead sinks Leaf topics Debug tf Unreachable Params

Plugins related to visualization.

Example RQT Perspective, where the plugins `rqt_service_caller`, `rqt_publisher`, `rqt_topic`, `rqt_plot` and `rqt_graph` are open and running

RQT - What is RQT?

- ① **RQT:** (ROS Qt-based GUI Framework) is a graphical user interface framework in ROS that provides a collection of pluggable tools for visualizing, debugging, and monitoring ROS applications.
- ② Key Features and Benefits:
 - **Graphical Interface:** offers a user-friendly GUI environment for tasks that might otherwise require command-line tools, simplifying data visualization and interaction
 - **Plugin Architecture:** modular design allows easy addition and management of tools as plugins, creating a centralized platform for debugging and analysis
 - **Customization:** developers can create their own RQT plugins using Python or C++, enabling tailored tools for specific needs
 - **Integration with ROS2:** tightly integrated with ROS2, facilitating seamless interaction with ROS2 nodes, topics, and services for real-time analysis

RQT - RQT plugins

Some of the most common RQT plugins are:

- ➊ **rqt_publisher**: GUI plugin for publishing arbitrary messages with fixed or computed field values
- ➋ **rqt_topic**: GUI plugin for inspecting ROS2 topics, showing topic names, types, latest values, bandwidth, and publishing rate
- ➌ **rqt_service_caller**: GUI plugin for calling arbitrary services
- ➍ **rqt_graph**: Visualizes the network of ROS2 nodes and topics, helping to understand the robot's computation graph
- ➎ **rqt_plot**: Creates visual representations of data from ROS2 topics, allowing for easier data analysis
- ➏ **rqt_tf_tree**: Visualizes the TF (Transform) Frame tree, showing the coordinate frames of a robot system and their relationships in a hierarchical structure in real time
- ➐ **rqt_image_view**: Displays images being published on ROS2 topics

We are going to showcase these plugins through some simple examples.

RQT - RQT plugins - rqt_publisher

Turtlesim Example

We run the turtlesim example:

```
ros2 run turtlesim turtlesim_node  
ros2 run turtlesim turtle_teleop_key
```

Command to start rqt_publisher

```
ros2 run rqt_publisher rqt_publisher
```

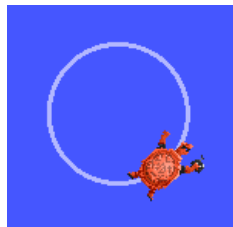
rqt_publisher_Publisher - rqt

Message Publisher

Topic: Type: Freq. Hz

topic	type	rate	expression
☒ /turtle1/cmd_vel	Twist	1.00	
angular	geometry_msgs/Vector3		
x	double	0.0	
y	double	0.0	
z	double	0.5	
linear	geometry_msgs/Vector3		
x	double	0.5	
y	double	0	
z	double	0.0	

rqt_publisher plugin



turtle view

RQT - RQT plugins - rqt_topic

Turtlesim Example

We run the turtlesim example:

```
ros2 run turtlesim turtlesim_node  
ros2 run turtlesim turtle_teleop_key
```

Command to start rqt_topic

```
ros2 run rqt_topic rqt_topic
```

The screenshot displays the `rqt_topic__TopicPlugin - rqt` window. On the left, the 'Topic Monitor' panel lists several topics. The `/turtle1/cmd_vel` topic is expanded, showing its `geometry_msgs/Twist` structure with fields for angular and linear velocities. The `/turtle1/color_sensor` and `/turtle1/pose` topics are also expanded, showing their respective data types and values. On the right, a 'Turtle view' shows a green turtle on a blue background, with a white circular path. A yellow arrow points from the turtle's current position to the 'Pose' data in the `/turtle1/pose` topic, which is highlighted in a box.

Topic	Type	Value
☐ /parameter_events	rcl_interfaces/msg/ParameterEvent	not monitored
☐ /rosout	rcl_interfaces/msg/Log	not monitored
☒ /turtle1/cmd_vel	geometry_msgs/msg/Twist	
angular	geometry_msgs/Vector3	
x	double	0.0
y	double	0.0
z	double	0.5
linear	geometry_msgs/Vector3	
x	double	0.5
y	double	0.0
z	double	0.0
☒ /turtle1/color_sensor	turtlesim/msg/Color	
b	uint8	255
g	uint8	184
r	uint8	179
☒ /turtle1/pose	turtlesim/msg/Pose	
angular_velocity	float	0.5
linear_velocity	float	0.5
theta	float	-1.067304015159607
x	float	4.666208744049072
y	float	6.0584516525268555

rqt_topic plugin with turtle view

RQT - RQT plugins - rqt_service_caller

Turtlesim Example

We run the turtlesim example:

```
ros2 run turtlesim turtlesim_node  
ros2 run turtlesim turtle_teleop_key
```

Command to start rqt_service_caller

```
ros2 run rqt_service_caller rqt_service_caller
```

The screenshot shows the `rqt_service_caller_ServiceCaller` window on the left and the `Turtlesim` window on the right. In the `rqt_service_caller` window, the 'Service' dropdown is set to `/spawn`, and the 'Call' button is highlighted with a red box. Below the 'Request' section, a table lists the arguments for the `/spawn` service:

Topic	Type	Expression
<code>/spawn</code>	<code>turtlesim/srv/Spawn</code>	
<code>x</code>	<code>float</code>	<code>8</code>
<code>y</code>	<code>float</code>	<code>8</code>
<code>theta</code>	<code>float</code>	<code>0.78</code>
<code>name</code>	<code>string</code>	<code>turtle2</code>

Below the 'Response' section, a table shows the response data:

Field	Type	Value
<code>/</code>	<code>turtlesim/srv/Spawn.Response</code>	
<code>name</code>	<code>string</code>	<code>'turtle2'</code>

The `'turtle2'` value in the response table is highlighted with a blue box. A green arrow points from this box to the `Turtlesim` window, where a new turtle (labeled `turtle2`) is visible at the coordinates (8, 8) with a heading of 0.78 radians. The original turtle is circled in white.

rqt_service_caller plugin with turtle view

RQT - RQT plugins - rqt_graph

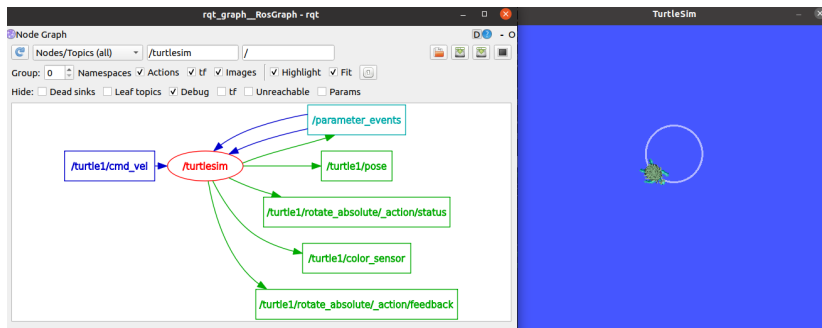
Turtlesim Example

We run the turtlesim example:

```
ros2 run turtlesim turtlesim_node  
ros2 run turtlesim turtle_teleop_key
```

Command to start rqt_graph

```
ros2 run rqt_graph rqt_graph
```



rqt_graph plugin with turtle view

RQT - RQT plugins - rqt_plot

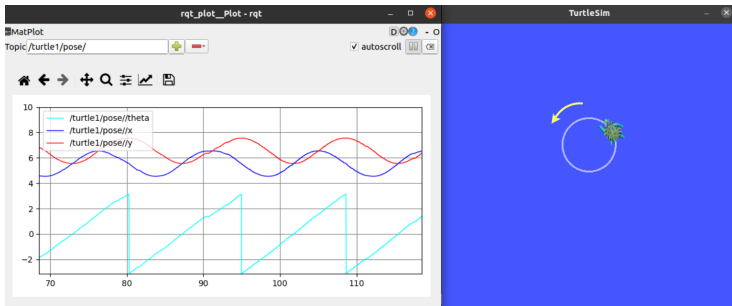
Turtlesim Example

We run the turtlesim example:

```
ros2 run turtlesim turtlesim_node  
ros2 run turtlesim turtle_teleop_key
```

Command to start rqt_plot

```
ros2 run rqt_plot rqt_plot
```



rqt_plot plugin with turtle view

RQT - RQT plugins - rqt_tf_tree

Double Pendulum Example

We run the double pendulum example:

```
ros2 launch pendu_control_pkg pendu_display_launch.py
```

Command to start rqt_tf_tree

```
ros2 run rqt_tf_tree rqt_tf_tree
```

The screenshot displays the `rqt_tf_tree` plugin interface. On the left, a tree structure shows the hierarchy of the double pendulum system:

- `base_link`
 - `joint1_sphere`
 - `link1`
 - `joint2_sphere`
 - `link2`

Each node in the tree provides metadata such as the broadcaster, average rate, buffer length, most recent transform, and oldest transform. On the right, a 3D visualization of the double pendulum is shown, with the base link in blue, the first joint in red, and the second joint in green. The interface also includes a 'Selection' panel with a list of links and joints, and a 'Displays' panel with checkboxes for various visual elements like status, collision, and update interval.

*rqt_tf_tree plugin with
double pendulum RViz2
view*

RQT - RQT plugins - rqt_image_view

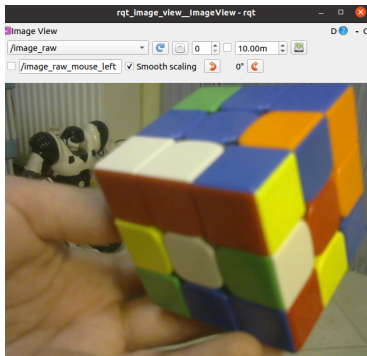
Web Camera Example

We run the web camera example:

```
ros2 run v4l2_camera v4l2_camera_node --ros-args -p  
video_device:=/dev/video0
```

Command to start rqt_image_view

```
ros2 run rqt_image_view rqt_image_view
```



rqt_image_view plugin with camera feed

URDF files - What is a URDF file?

- ① **URDF** (Unified Robot Description Format) is an XML format used in ROS2 to describe a **robot's physical configuration and properties**; its links/joints structure, kinematics, inertial characteristics, and visual/collision properties. It can be loaded in visualization tools like **RViz2** and in simulation tools like **Gazebo** in ROS2.
- ② As an **XML file**, it is written using **tags**.
- ③ It often takes a set of **meshes** (usually .stl or .dae files) and describes the transformations to one another (both position and orientation), in order to shape and build the desirable robotic system.
- ④ It describes a **robot as a tree** (or graph) of **links** (rigid bodies) connected by **joints**. The structure follows a simple chain-like logic, using this basic building block to model the entire robot:

parent_link → **joint** → **child_link**

URDF files - Creating a URDF File

The key components of a URDF file are:

- 1 **Links:** represent the rigid bodies of the robot, each typically containing visual, collision, and inertial elements, respectively describing how it looks, interacts, and moves physically
- 2 **Joints:** define the connections between links (parent $\rightarrow$ child) and specify the type of motion (revolute, prismatic, fixed, etc.)
- 3 **Visuals:** specify how each link appears in visualization tools, including geometry, color, and materials
- 4 **Collisions:** define the geometry used for simulation and collision detection, often simplified for efficiency
- 5 **Meshes:** external 3D models (e.g., .dae, .stl files) can be referenced for detailed visual or collision geometry

```
<?xml version="1.0" ?>
<robot name="myarm_300_pi">

  <link name="base">
    <visual>
      <geometry>
        <mesh filename="package://myarm_300_pi/meshes/base.dae"/>
      </geometry>
      <origin rpy=" 0 0 0" xyz="0.0 0 0" />
    </visual>
    <collision>
      <geometry>
        <mesh filename="package://myarm_300_pi/meshes/base.dae"/>
      </geometry>
      <origin rpy=" 0 0 0" xyz="0.0 0 0" />
    </collision>
  </link>

  <joint name="joint1" type="revolute">
    <axis xyz="0 0 1"/>
    <limit effort="1000.0" lower="-2.7925" upper="2.7925" velocity="0"/>
    <parent link="base"/>
    <child link="link1"/>
    <origin rpy="0 0 0" xyz="0 0 0.1695"/>
  </joint>

  <link name="link1">
    <visual>
      <geometry>
        <mesh filename="package://myarm_300_pi/meshes/j1.dae"/>
      </geometry>
      <origin rpy=" 0 0 0" xyz="0.0 0 -0.086" />
    </visual>
    <collision>
      <geometry>
        <mesh filename="package://myarm_300_pi/meshes/j1.dae"/>
      </geometry>
      <origin rpy=" 0 0 0" xyz="0.0 0 -0.086" />
    </collision>
  </link>
```

Main components of a URDF file

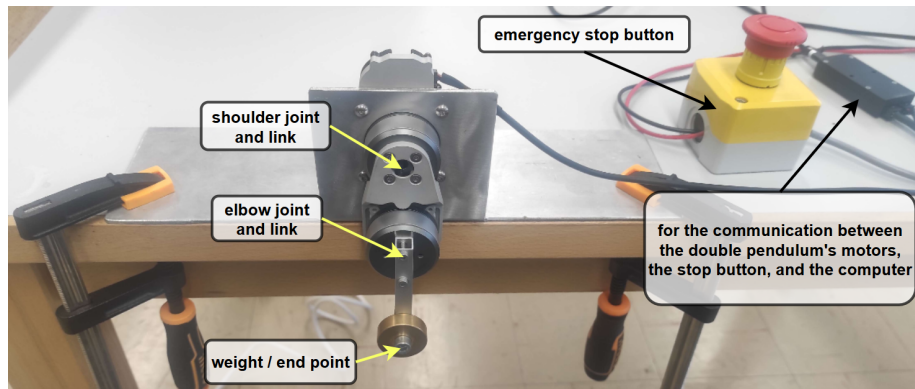
URDF files - What is a XACRO file?

- 1 **XACRO** (XML Macros) is an **XML macro language**. It's used to construct shorter and more readable XML files by using **macros** that expand to larger XML expressions.
- 2 It's a **preprocessor for URDFs**, letting us write *modular* and *reusable* robot descriptions with macros, properties, loops, and conditionals.
- 3 We can convert a `.xacro` file to a `.urdf` file using the **xacro** command in ROS2.

XACRO command for a file called `robot.xacro`

```
xacro robot.xacro > robot.urdf
```

Double Pendulum Exercise



The double pendulum has 2 joints: the shoulder and the elbow joint (it can be thought of as a 2-DOF robotic arm too). We want to continuously visualize its configuration using RViz2. We also want to create a simple impedance PD controller that will be able to move the double pendulum to each one of its 4 equilibrium points.

Double Pendulum Exercise - Create URDF

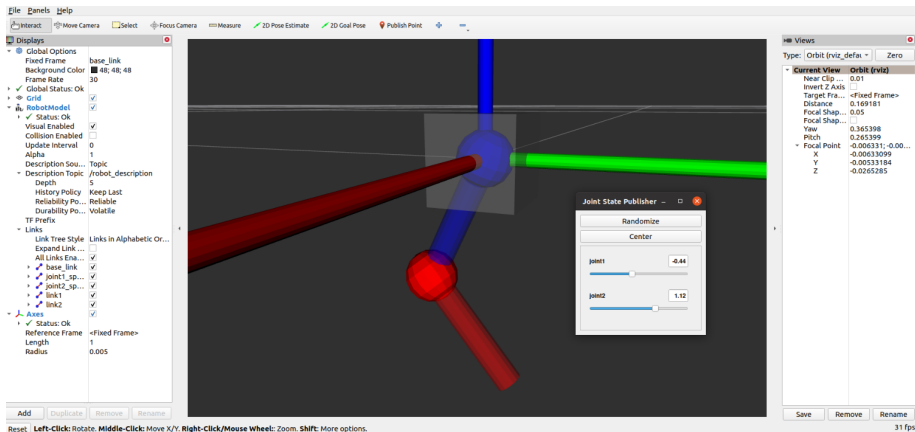
To describe the double pendulum, we create its URDF file from scratch. The **links** are modeled as cylindrical rods, and the revolute **joints** as simple spheres. Collision elements are not required, since the pendulum's links move independently and rotate freely through a full 360° without interference.

1 Analytical Structure:

- **Base Link:** gray cube (**base_link**) as fixed base (our fixed frame)
- **First Joint:** **joint1** (revolute, x-axis) connects base to **joint1_sphere** link
- **First Link:** **joint1_sphere** (blue sphere) + **link1** (blue cylinder, 5 cm), *connected through a fixed **joint1_link** joint*
- **Second Joint:** **joint2** (revolute, x-axis) connects **link1** to **joint2_sphere** link
- **Second Link:** **joint2_sphere** (red sphere) + **link2** (red cylinder, 5 cm), *connected through a fixed **joint2_link** joint*

2 Summary: **base_link** → **joint1** → **link1** → **joint2** → **link2**

Double Pendulum Exercise - Visualize using RViz2

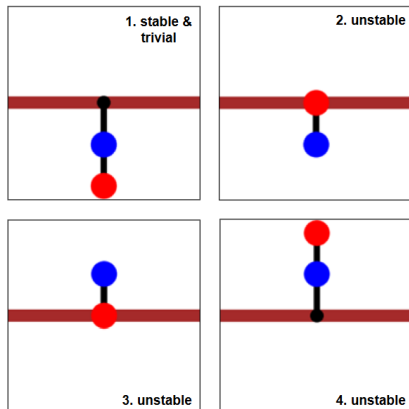


Visualization of the Double Pendulum in RViz2

Double Pendulum Exercise - Visualize using RViz2 - Our Executable Node

The node `double_pendulum_node` monitors and controls the double pendulum through two MD80 motor drives interfaced via `pyCandle` (to install type `'pip3 install pyCandleMAB'`).

- 1 Publisher:** Publishes the angles and angular velocities of both joints (double pendulum's current state) on the topic `/joint_states` (message type: `sensor_msgs/JointState`) at 100 Hz, allowing real-time monitoring and visualization in RViz2.
- 2 Subscriber:** Listens to the topic `/equilibrium` (message type: `std_msgs/Int32`) to receive target equilibrium configurations (1–4, as shown in the picture) and sets the corresponding joint angles by commanding the motors to their respective target positions.



Equilibrium Points

Double Pendulum Exercise - Visualize using RViz2 - Launch File Overview

The launch file launches three executable nodes and loads automatically the double pendulum model and RViz2 configuration. Its key points are:

① Executable Nodes:

- `robot_state_publisher`: reads the robot's URDF description (stores it in `robot_description` parameter) and publishes the transforms of all links to the TF tree
- `rviz2`: starts RViz2, loading the configuration file specified by `rviz_config_file`
- `double_pendulum_node`: runs our custom control node that interfaces with the physical motors and publishes joint states

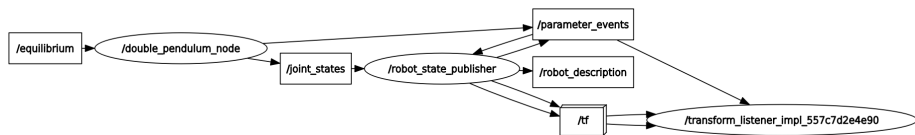
② URDF File: The `robot_description` parameter is generated from the double pendulum's URDF, providing the robot's structure (joints and links) and visuals

③ RViz2 Config File: The `rviz_config_file` defines RViz2's display layout and visualization parameters

Double Pendulum Exercise - Inspect ROS2 Graph

Use RQT to inspect the ROS2 graph

```
ros2 run rqt_graph rqt_graph
```



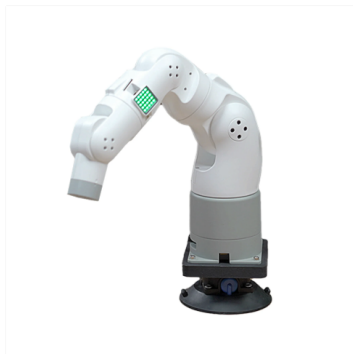
Double Pendulum's ROS2 Graph

Double Pendulum Exercise - Explaining ROS2 TF (Transform) Frame Tree

RViz2 doesn't subscribe directly to `/joint_states`, instead, it visualizes the robot based on the `/tf transforms` produced by the node `robot_state_publisher` (which in turn, as we said, is subscribed to topic `/joint_states` and uses the parameter `robot_description`).

- ❶ **What is tf?** It is ROS2's transform system that **tracks all coordinate frames** (links and joints) over time and answers "Where is frame A relative to frame B right now?"
- ❷ **/tf vs /tf_static:** `/tf` publishes **dynamic transforms** for moving joints and robot motion continuously, while `/tf_static` publishes **fixed transforms** for rigid joints only once.
- ❸ **Double Pendulum Exercise:** `/tf` carries `joint1` and `joint2` angles, `/tf_static` carries fixed offsets such as `sphere` → `link`.

Elephant robotic arm Exercise



MyArm 300 Pi 7-DOF



MyCobot 280 Pi 6-DOF

These are the two platforms we have in lab. We are going to work on the MyArm 7-DOF robotic arm, that carries one more degree of freedom (the redundancy offers more kinematic capabilities) and a better on-board RPi4 computer. We want to continuously visualize its configuration in RViz2, while we control its 7 revolute joints with GUI sliders.

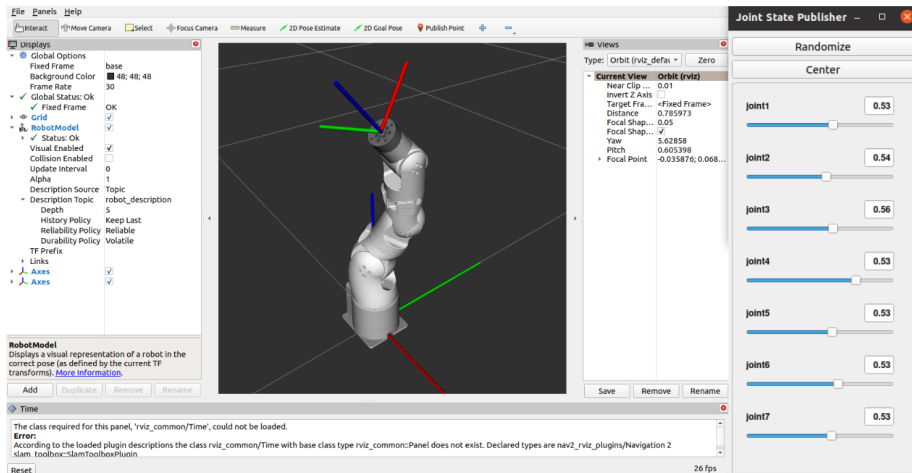
Elephant robotic arm Exercise - Create URDF

- 1 The URDF for our elephant's robotic arm, contains both visual and collision elements. We use .dae meshes files to make a representation as accurate as possible to the real physical system. There are 9 **links** and 8 **joints** (7 revolute -the ones we control- & 1 fixed), that follow this hierarchical chain:

base → joint1 → link1 → joint2 → link2 → joint3 → link3
→ joint4 → link4 → joint5 → link5 → joint6 → link6 →
joint7 → link7 → link7_to_endeffector → endeffector

- 2 The **endeffector** is a dummy link, acting as a flexible reference frame for potential tools (a gripper, a suction cup, or other manipulators), with no visual or collision geometry.

Elephant robotic arm Exercise - Visualize using RViz2



Visualization of the Elephant MyArm in RViz2

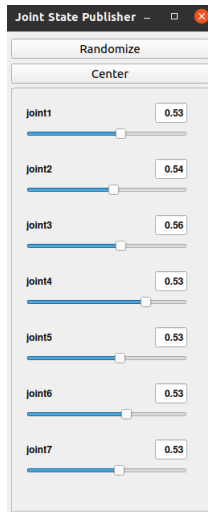
Elephant robotic arm Exercise - Visualize using RViz2 - Our Executable Node

The node `simple_control_node` connects to the Elephant MyArm 300 PI robotic arm through a serial interface (port and baudrate configurable via ROS2 parameters) and initializes it to a zero configuration.

- ❶ **Publisher:** Continuously publishes the current joint positions of all 7 arm joints on the topic `/joint_states` (message type: `sensor_msgs/JointState`) at 100 Hz, providing real-time feedback of the arm's configuration for visualization and monitoring.
- ❷ **Subscriber:** Listens to the topic `/joint_states_targets` (message type: `sensor_msgs/JointState`) to receive desired joint positions from GUI sliders, and commands the robotic arm to move accordingly, respecting the configured joint speed (parameter `joint_speed`).

Elephant robotic arm Exercise - Visualize using RViz2 - Launch File Overview

The launch file follows a structure similar to the one used in the double pendulum exercise, with one key addition: a fourth executable node from the package `joint_state_publisher_gui`. This node reads the robot's description from the `/robot_description` parameter and provides **interactive GUI sliders** for manually controlling each joint of the robotic arm. By default, it publishes joint values to the topic `/joint_states`, but for our purposes the topic is remapped to `/joint_states_targets`, in order to separate user inputs from the robot's actual state.

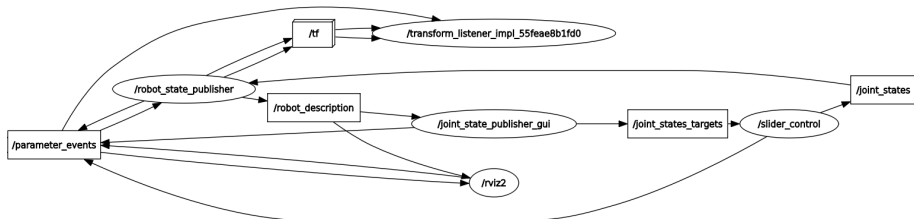


*joint_state_publisher_gui
sliders*

Elephant robotic arm Exercise - Inspect ROS2 Graph

Use RQT to inspect the ROS2 graph

```
ros2 run rqt_graph rqt_graph
```

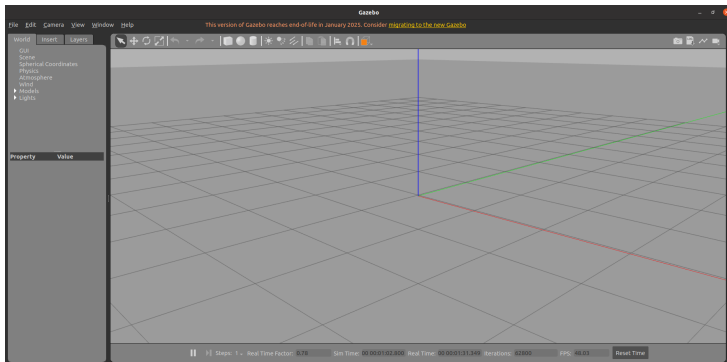


Elephant MyArm's ROS2 Graph

An interactive closed loop is created: GUI sliders → /joint_states_targets → slider_control → robot → /joint_states → RViz2 visualization

Gazebo mention

Gazebo is a **robot simulator**. The difference between it and RViz was summed up by Morgan Quigley (one of the original ROS developers) in his book "Programming Robots with ROS": *"... rviz shows you what the robot thinks is happening, while Gazebo shows you what is really happening ..."*.



Gazebo starting Environment

- **ROS Home**
- **ROS2 Documentation for all distros, here for Foxy**
- **RQT documentation & RQT overview for ROS2**
- **RQT Plugins**
- **URDF files explainer video**
- **ROS wikis about URDF's links and joints**
- **ROS wiki about XACRO files**
- **RViz User Guide**
- **Console tool for configuring and setting up md80 via CANdle**
- **Elephant Robotics Documentation**